

НАУЧНО-ИССЛЕДОВАТЕЛЬСКАЯ РАБОТА

EraSeMap: проверяемое удаление персональных данных из распределённых систем и моделей машинного обучения

Единый алгоритм FIND–ERASE–PROVE

Автор: Нұрланұлы Дулат, 9 «Б» класс

КГУ «Специализированный лицей-интернат „Білім-инновация“»

Управления образования города Алматы

Направление II — математическое моделирование экономических и социальных процессов

Секция: информатика

Научный руководитель: Смағұл Ерзат Айдынұлы, учитель информатики

Алматы, 2026

Аннотация

Если удалить главную запись о человеке, это ещё не значит, что данные о нём исчезли из системы. Копия может остаться в кэше, в индексе поиска, в резервной копии. След может остаться в обученной модели — даже если саму запись давно стёрли. И самое неприятное: удалённое иногда возвращается — после восстановления из бэкапа, синхронизации или переразвёртывания модели.

Я захотел проверить, можно ли дать один честный ответ на вопрос «данные точно удалены?», а не пять разных ответов от пяти разных систем. Для этого я собрал алгоритм из трёх шагов: FIND ищет, где данные могли остаться; ERASE выбирает минимальный набор действий для их удаления, включая удаление влияния из модели — machine unlearning [1, 2]; PROVE проверяет, не вернутся ли данные после будущих операций с системой, и только после этого разрешает выдать сертификат. Алгоритм честно возвращает один из трёх ответов — COMPLETE_WITHIN_ENVELOPE, INCOMPLETE или UNVERIFIED — и никогда не выдаёт отсутствие доказательств за успех.

Я протестировал это на 60 сценариях с реальными сервисами (Keycloak, MLflow, Qdrant): мой алгоритм не дал ни одного ложного «удалено», тогда как проверка по обычному статусу сервиса ошиблась в 45 случаях из 60, а полный аудит узлов — в 5. На скрытых графах восстановления поиск нужного пути занял 7 попыток вместо 49 при переборе всех вариантов и 13 — при случайной стратегии; жадная стратегия дала такой же результат, как активный поиск, и я это не скрываю. В экспериментах с реальными сервисами (PostgreSQL, Redis, зашифрованный бэкап) выбранный план был в среднем примерно в 17 раз быстрее полной пересборки системы и писал почти на 95% меньше данных на диск — усреднено по 20 прогонам, с заметным разбросом между ними.

Отдельно я попробовал ускоренные методы удаления влияния из открытой модели Qwen2.5-1.5B. Они не прошли все заранее заданные критерии одновременно. Поэтому в пределах этого эксперимента точное переобучение адаптера осталось безопасным запасным вариантом. Это не универсальный вывод для всех моделей, а честный результат конкретной проверки. Работа не претендует на внедрение в FaceID или eGov: это контролируемый стенд с синтетическими данными, а не готовая производственная система.

Ключевые слова: проверяемое удаление, machine unlearning, data lineage, temporal replay, биометрия, fail-closed, сертификат удаления.

Abstract

Deleting the main record about a person does not mean the data is gone from the system. A copy can remain in a cache, a search index, or a backup. A trace can remain inside a trained model even after the original record is deleted. And deleted data sometimes comes back — after a backup restore, a sync, or a model redeploy.

I wanted to find out whether a system can give one honest answer to "is this person's data really gone?" instead of five different answers from five different components. I built a three-stage algorithm: FIND looks for places data could remain; ERASE picks the smallest sufficient set of actions to remove it, including removing the model's learned influence (machine unlearning [1, 2]); PROVE replays future operations and only then allows a certificate to be issued. The algorithm returns one of three honest answers — COMPLETE_WITHIN_ENVELOPE, INCOMPLETE, or UNVERIFIED — and never treats missing evidence as success.

Across a 60-case transfer test on real services (Keycloak, MLflow, Qdrant), EraSeMap produced zero false-COMPLETE verdicts, versus 45 for the plain service status and 5 for a full node audit. On hidden recovery graphs, active search needed 7 probes versus 49 for exhaustive search and 13 for random testing; a greedy baseline matched the active search, which I report rather than hide. In 20 paired real-service trials, the selected plan was on average roughly 17 times faster than a full rebuild and wrote almost 95% fewer bytes, with real variation between individual runs. Fast unlearning candidates on Qwen2.5-1.5B failed the combined thresholds. Within this experiment, exact adapter retraining remained the safe fallback; this is a bounded result, not a universal claim about every model.

1. Введение

Когда человек просит удалить его данные, система обычно отвечает одним словом: «готово». Но это «готово» почти никогда не значит того, что кажется. Одна и та же запись о человеке распадается на десятки копий: в кэше, в поисковом индексе, в резервной копии, в логах, в экспорте для аналитики. Если данные использовались для обучения модели — например, распознавания лиц, — модель могла «запомнить» человека даже после того, как исходную запись стёрли. Право на удаление персональных данных закреплено в статье 17 Общего регламента по защите данных Евросоюза [16] и в казахстанском Законе «О персональных данных и их защите» [17], но ни один из этих законов не говорит, как техническими средствами доказать, что удаление действительно произошло.

Я начал с малого: попробовал вручную удалить тестового «пользователя» из простой связки база данных + кэш + маленькая модель. Быстро выяснилось, что даже в такой игрушечной системе легко пропустить один канал — например, забыть, что модель уже «запомнила» удалённого человека, хотя запись о нём стёрта. Это и натолкнуло меня на мысль, что нужен не список инструкций «как удалять руками», а алгоритм, который сам проверяет, не забыли ли что-то, и не разрешает выдать положительный ответ, пока не проверены все возможные места, где данные могли остаться.

Существующие исследования решают только часть задачи. Data lineage описывает, откуда пришли данные [9]. Machine unlearning уменьшает влияние конкретных обучающих примеров на модель [1, 2], а работы по verification-of-unlearning пытаются понять, насколько этому можно доверять [3–8]. Патенты и внутренние политики компаний описывают, как отслеживать происхождение данных, бэкапы и подтверждения об удалении [10–12, 15]. Но ни один из этих подходов не связывает физические копии, производные данные, влияние в модели и возможность будущего восстановления в одном общем вердикте.

Вопрос, который я себе поставил: можно ли дать один проверяемый ответ на запрос об удалении данных конкретного человека — ответ, который одновременно учитывает физические копии, производные данные, влияние в модели и то, что данные могут вернуться после восстановления системы?

Что я сделал в этой работе:

- собрал единый трёхстадийный алгоритм FIND–ERASE–PROVE, который не объявляет удаление завершённым без доказательств;

- построил формальную модель, которая различает физические, производные и связанные с моделью данные;
- реализовал точный выбор минимального набора действий для удаления, с явным отказом, если удалить полностью невозможно;
- добавил проверку «во времени» — replay будущих операций, которая запрещает сертификат, если данные могут вернуться;
- сравнил быстрые методы machine unlearning с полным переобучением модели с нуля и не стал скрывать случаи, когда быстрый метод проигрывал;
- проверил всё это на нескольких уровнях: на скрытых графах, на реальных сервисах и на открытой модели Qwen2.5-1.5B.

2. Предшествующие работы и граница новизны



Рисунок 1. Один алгоритм EraSeMap: FIND, ERASE и PROVE. Составлено автором по протоколу EraSeMap.

Ничего из перечисленного я не изобретал сам — ни графы происхождения данных [9], ни machine unlearning [1, 2], ни саму идею проверки во времени. Моя часть работы — соединить всё это в одно правило принятия решения: система выдаёт положительный вердикт только тогда, когда проверены все обязательные каналы сразу, а не когда прошла успешно одна локальная проверка.

Разница с обычным аудитом происхождения данных в том, что для меня важен не только тип узла в графе, а есть ли реальный пригодный путь от конкретного человека к активному артефакту. Разница с «чистым» machine unlearning в том, что удаление влияния из модели закрывает только модельный канал — бэкап, кэш и поисковый индекс при этом могут остаться нетронутыми, и я проверяю их отдельно. Разница со снимком состояния

системы в том, что «сейчас пусто» не доказывает, что после следующего восстановления объект не появится снова, — поэтому в алгоритме есть отдельная стадия PROVE. И, наконец, я не даю универсального обещания «данные удалены навсегда»: вердикт COMPLETE_WITHIN_ENVELOPE прямо ограничен той системой и теми переходами, которые были зарегистрированы и проверены.

3. Модель системы

3.1 Типизированный граф

Систему я представил в виде графа $G = (V, E, \tau, s)$, где V — это артефакты и сервисы (файлы, записи в базе, индексы, копии модели), E — операции происхождения данных или их восстановления, τ — тип узла или ребра, а s — привязка к конкретному человеку (subject binding). Для человека u множество путей, по которым его данные ещё могут быть доступны, я обозначил $R_u(G)$. Путь считается активным, если конечный артефакт на этом пути может хранить, выдавать, распознавать или восстанавливать информацию об этом человеке.

Я выделил семь обязательных каналов проверки C_u :

- physical — исходные и резервные физические артефакты;
- derivative — шаблоны, эмбединги, индексы, экспорты;
- model — влияние, оставшееся в обученной модели;
- privacy — заранее объявленные privacy-проху проверки;
- utility — качество работы системы для остальных пользователей и задач;
- temporal — отсутствие возврата данных после будущих операций;
- coverage — достаточность зарегистрированного мониторинга системы.

Каждая проверка возвращает один из трёх результатов: PASS, FAIL или UNKNOWN. Я сознательно не стал ограничиваться двумя значениями «да/нет» — если проверка не может точно ответить, она обязана сказать «не знаю», а не гадать.

3.2 Три вердикта

Финальный ответ — тоже один из трёх вариантов:

- COMPLETE_WITHIN_ENVELOPE — когда $R_u(G)$ пусто, каждый обязательный канал прошёл проверку, доказательства поиска валидны и проверка во времени подтвердила безопасность;
- INCOMPLETE — когда найден хотя бы один конкретный активный или потенциально восстанавливаемый путь;

- UNVERIFIED — когда доступных доказательств недостаточно ни для утверждения об удалении, ни для утверждения об остатке.

Третье значение — не техническая деталь для полноты картины, а осознанное решение: отсутствие доказательств никогда не должно превращаться в «успех» только потому, что так удобнее.

4. Единый алгоритм EraSeMap

4.1 FIND

FIND сначала проверяет уже зарегистрированный граф системы. Если есть подозрение, что карта системы неполная, включается вторая часть: адаптер запускает безопасные тестовые операции восстановления и смотрит, где и когда «всплывает» тестовый субъект. После каждой такой пробы остаются только те гипотезы о структуре графа, которые совпадают с результатом наблюдения. Формально: пусть H — конечное пространство версий графа, а $Obs(q, h)$ — результат эксперимента q при гипотезе h . После наблюдения o остаётся подмножество $H' = \{h \in H : Obs(q, h) = o\}$.

Для выбора следующей пробы я использовал точный одношаговый minimax: выбирается допустимая проба, которая гарантированно уменьшает худший возможный оставшийся класс гипотез; при равенстве вариантов используется фиксированное правило приоритета. Это гарантия только для одного следующего шага, а не обещание, что весь путь поиска будет оптимальным, — я специально это не завишаю.

4.2 ERASE

Пусть A — конечный список разрешённых действий, $c(a)$ — стоимость действия a , а $Apply(G, B)$ — состояние системы после применения подмножества действий $B \subseteq A$. План допустим, если все действия в нём разрешены и повторная проверка (replay) закрывает каждый активный путь и каждый обязательный канал. Дальше решается задача выбора минимального достаточного набора действий:

$$B^* = \arg \min \sum c(a), \text{ при условии } Feasible(B) = true$$

При равенстве нескольких вариантов используется фиксированный лексикографический порядок. Если полного допустимого плана не существует, алгоритм честно возвращает INCOMPLETE или UNVERIFIED — но никогда не выдаёт «частичную квитанцию» вместо полного ответа.

Отдельно в ERASE входит модельная ветвь: я сравнивал быстрый метод удаления влияния с полным переобучением модели сразу по нескольким критериям — насколько

хорошо забыты удалённые примеры, не пострадало ли качество для остальных пользователей, не появляется ли влияние снова после перезагрузки модели. Модель считается прошедшей проверку, только если выполнены все критерии одновременно — провал хотя бы одного означает, что нужно полное переобучение.

4.3 PROVE

Пусть q_0 — состояние системы сразу после ERASE, а δ — набор зарегистрированных будущих операций (восстановление, синхронизация и так далее).
Условие безопасности:

$$\text{Safe}(u) \Leftrightarrow \forall q \in \text{Reach}(q_0, \delta): \text{Residual}(u, q) = \text{false}$$

Если повторное выполнение операций находит след возврата данных, PROVE возвращает INCOMPLETE вместе с конкретным контрпримером — какая операция и в какой момент вернула данные. Если неизвестно, все ли будущие переходы учтены, результат — UNVERIFIED. Сертификат выдаётся только после того, как эта проверка пройдена.

4.4 Общая формула

Пусть P — закрытие физических и модельных путей, D — валидность доказательств поиска, T — безопасность во времени. Тогда:
 $\text{COMPLETE_WITHIN_ENVELOPE} \Leftrightarrow P \wedge D \wedge T$.

5. Формальные свойства

Часть утверждений я проверил формально в Lean, без «sorry» — то есть без недоказанных мест, спрятанных за заглушкой. Доказаны четыре условных свойства:

- если топология системы известна полностью и локальные проверки надёжны, то после replay-проверки не остаётся ни одного из представленных путей утечки;
- точный конечный алгоритм выбора плана действий не может стоить дороже любого другого допустимого плана из того же списка;
- если снимок состояния показывает пустоту, это переносится на все достижимые зарегистрированные состояния — при условии полного покрытия мониторинга;
- точный алгоритм временной проверки безопасен и минимален — при явном условии выполнения feasibility и soundness.

Отдельно я построил контрпримеры, которые показывают обратное: без полного покрытия топологии скрытый остаток данных возможен даже при пройденных проверках, а прохождение канала без гарантии его надёжности вообще ничего не доказывает. Python-реализацию я отдельно сверил с полным перебором. Важно понимать: доказательства в Lean не доказывают, что сам Python-код или адаптеры к реальным сервисам работают правильно, и не доказывают полноту организации в целом — это осознанное ограничение работы, а не то, что я упустил.

6. Реализация

Публичная точка входа `run_erasemap` в моём Python-коде возвращает результаты всех трёх стадий и общий вердикт. Адаптеры к сервисам я специально отделил от чистой логики проверки — так алгоритм не выполняет неразрешённые сетевые или разрушающие операции без явного разрешения. Каждый набор доказательств связан хешами и, где это предусмотрено протоколом, подписан по схеме Ed25519. Через CLI можно собрать офлайн-проверяемую демонстрацию и отчёты.

Внутри отчётов и формальной части я использовал несколько внутренних сокращений (PCUG, GhostGraph, CDC, RSE, MSC) — это не пять разных алгоритмов, а рабочие названия отдельных проверок внутри одного EraSeMap; для итогового результата это деталь реализации, а не отдельные достижения.

7. Как проводилась проверка

7.1 Основная метрика

Главная метрика — доля ложных «удалено»:

$$FCR = (\text{число ложных COMPLETE}) / (\text{число случаев, где данные реально не были удалены})$$

Дополнительно я считал: число проб при поиске, стоимость действий, время выполнения, объём записанных байт, потери в качестве для оставшихся пользователей, случаи возврата данных, расхождения с полным перебором, степень «забывания» в модели и разницу в приватности.

7.2 Уровни проверки

- Тест механизмов — разница между аудитом только по типу узла и проверкой реального пригодного пути.

- Перенос на готовые сервисы — 60 случаев на зафиксированных версиях Keycloak, MLflow и Qdrant.
- Скрытые графы — сравнение активного поиска, жадной стратегии, случайной стратегии и полного перебора на ограниченном наборе.
- Реальные процессы — сравнение выбранного плана ERASE с полной пересборкой на PostgreSQL, Redis и зашифрованном бэкапе.
- Временная лаборатория — проверка отложенного восстановления, синхронизации, пересборки кэша и индекса, а также случаев неполной регистрации системы.
- Опыты с лицами — ограниченное удаление влияния и проверка качества для оставшихся пользователей.
- Qwen-TOFU — влияние обучающих примеров на реально доступной модели 1.5B.

Протоколы, начальные значения (random seeds), пороги и хеши исходного кода я зафиксировал заранее, до финальных прогонов. Неудачные результаты я сохранил и не переписывал задним числом — это было для меня принципиально важно.

7.3 Три примера, которые проще всего показать вживую

Кроме таблиц с цифрами, у меня есть три конкретных прогона, которые я могу просто показать и объяснить на пальцах — без формул.

Пример 1 (PostgreSQL). Я поднял настоящую базу данных PostgreSQL, создал запись о человеке, отдельную «производную» таблицу и резервную копию (дамп). Потом удалил запись из главной таблицы. EraSeMap проверил систему и ответил INCOMPLETE — с точным путём postgres-derived, то есть «человек всё ещё виден через производную таблицу». Я убрал и производную таблицу, и дамп — только после этого алгоритм сказал COMPLETE. Здесь легко показать разницу: обычная проверка «строка удалена из главной таблицы?» сказала бы «да, всё готово» и ошиблась бы [14].

Пример 2 (возврат после бэкапа). Я 20 раз повторил один и тот же сценарий: снимок системы выглядел полностью пустым — ни записи, ни кэша, ни модели. Но при восстановлении из зашифрованного бэкапа человек каждый раз появлялся заново. Мой алгоритм поймал это все 20 из 20 раз и выбрал дешёвое решение — не удалять весь бэкап целиком (это дорого), а поставить своего рода «метку», которая гасит запись при восстановлении. После этого при повторных проверках возвратов не было ни разу (0 из

20). Это самый простой способ объяснить, зачем вообще нужна стадия PROVE: пустой снимок сейчас — не то же самое, что «данные не вернутся никогда» [14].

Пример 3 (демонстрация одной командой). В моём проекте есть готовый тестовый пример, где у «удалённого» человека остался один активный шаблон лица. Одна команда в терминале — `erasemap audit examples/five_branch_system.json --subject subject-1` — запускает проверку и сразу возвращает INCOMPLETE с путём `source → template`. Это можно показать вживую за полминуты прямо на защите, без объяснения формул: запустил команду — увидел честный ответ [14].

8. Результаты

Проверка	EraSeMap	Baseline	Интерпретация
Ложный COMPLETE на сервисах (60 случаев)	0/60	аудит узлов — 5/60; обычный статус сервиса — 45/60	проверяется реальный путь, а не просто ответ команды
Число проб для скрытого графа	7	жадная — 7; случайная — 13; полный перебор — 49	выигрыш против случайной стратегии и перебора; с жадной — ничья, это не скрыто
Совпадение точного решателя действий с перебором	3072 / 3072	полный перебор	расхождений в проверенной ограниченной задаче нет
Время выполнения плана (среднее по 20 прогонам)	≈ 5,7% от rebuild-all	полная пересборка — 100%	план в среднем примерно в 17 раз быстрее; разброс между прогонами заметный
Записанные байты (среднее по 20 прогонам)	≈ 5,4% от rebuild-all	полная пересборка — 100%	в среднем почти на 95% меньше записанных данных
Поиск временных рисков	30 / 30 найдено	снимок состояния — 0 / 30 найдено	снимок «сейчас пусто» не видит будущих возвратов
Возврат данных после контроля	0 / 30	без контроля — 30 / 30	выбранные защитные действия закрыли все найденные риски
Совпадение временного решателя с перебором	16 384 / 16 384	полный перебор	расхождений в ограниченной задаче нет

По итогам 20 парных экспериментов с реальными сервисами все локальные процессы завершились успешно, данные остальных пользователей не пострадали ни разу. В проверке переноса на 60 сценариев также не было ни потери нужных данных, ни возвратов после контроля.

Здесь я хочу сразу сказать честно: у меня было 20 повторов эксперимента с реальными процессами, и цифры «в среднем в 17 раз быстрее» и «почти на 95% меньше байт» — это средние значения по этим 20 прогонам, а не результат одного удачного запуска. Разброс между отдельными прогонами был заметным — от «примерно в 12 раз

быстрее» до «более чем в 20 раз быстрее», в зависимости от того, какие именно сервисы участвовали в конкретном тесте. На таком небольшом числе повторов я не считаю правильным давать более точную цифру, чем «примерно на порядок быстрее».

8.1 Модельный канал

В опытах с лицами я получил положительные результаты по данным, подготовленным для проекта, включая последовательную проверку качества для оставшихся пользователей. Подчёркиваю: это не сертификация приватности в юридическом смысле, а внутренняя техническая проверка.

Первую версию быстрого метода на Qwen (Qwen v1) я проверил на трёх начальных значениях — и она провалилась: результат приближался к точному переобучению по части метрик, но одновременно не прошёл критерии забывания и качества для остальных данных. Во второй версии (Qwen v2) я отделил подбор параметров от финальной проверки на пяти новых начальных значениях. Новый вариант был минимум в 30,48 раза быстрее и не показал возврата влияния после перезагрузки, но слишком сильно затронул другие, не связанные с удалением примеры и не прошёл часть проверок совпадения и качества. Поэтому итоговый результат — тоже FAIL, а точное переобучение адаптера осталось безопасным запасным вариантом в рамках этой проверки.

Мне не понравилось, что быстрый метод не сработал, — я рассчитывал на другой результат. Но именно поэтому я оставил переобучение как запасной вариант, а не подогнал пороги под удобный итог.

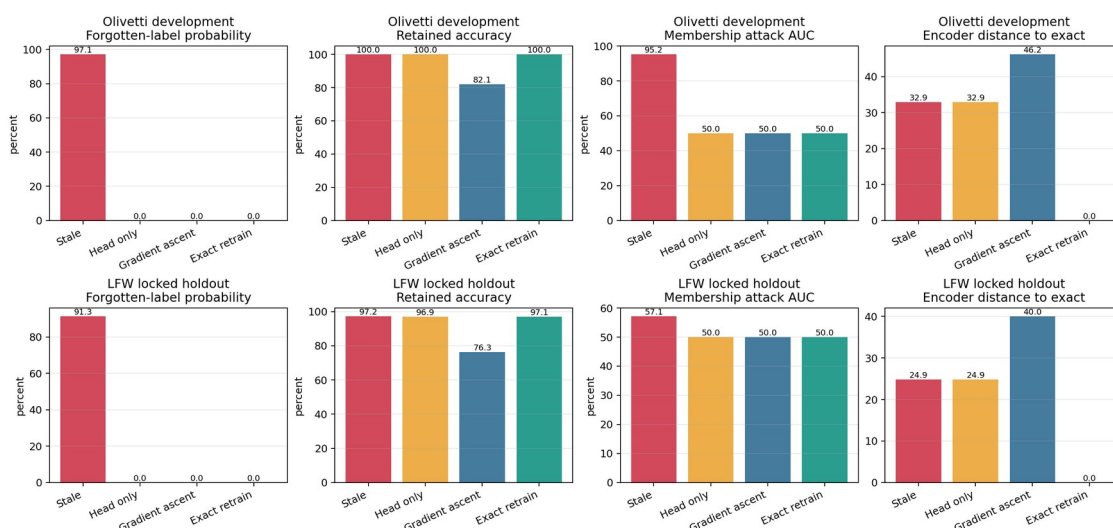


Рисунок 2. Сравнение четырёх стратегий после удаления (Olivetti Faces и LFW): забывание, точность для оставшихся, membership-атака, дистанция до эталонной модели. Источник: репозиторий проекта, docs/assets/unlearning-comparison.png [14].

9. Обсуждение

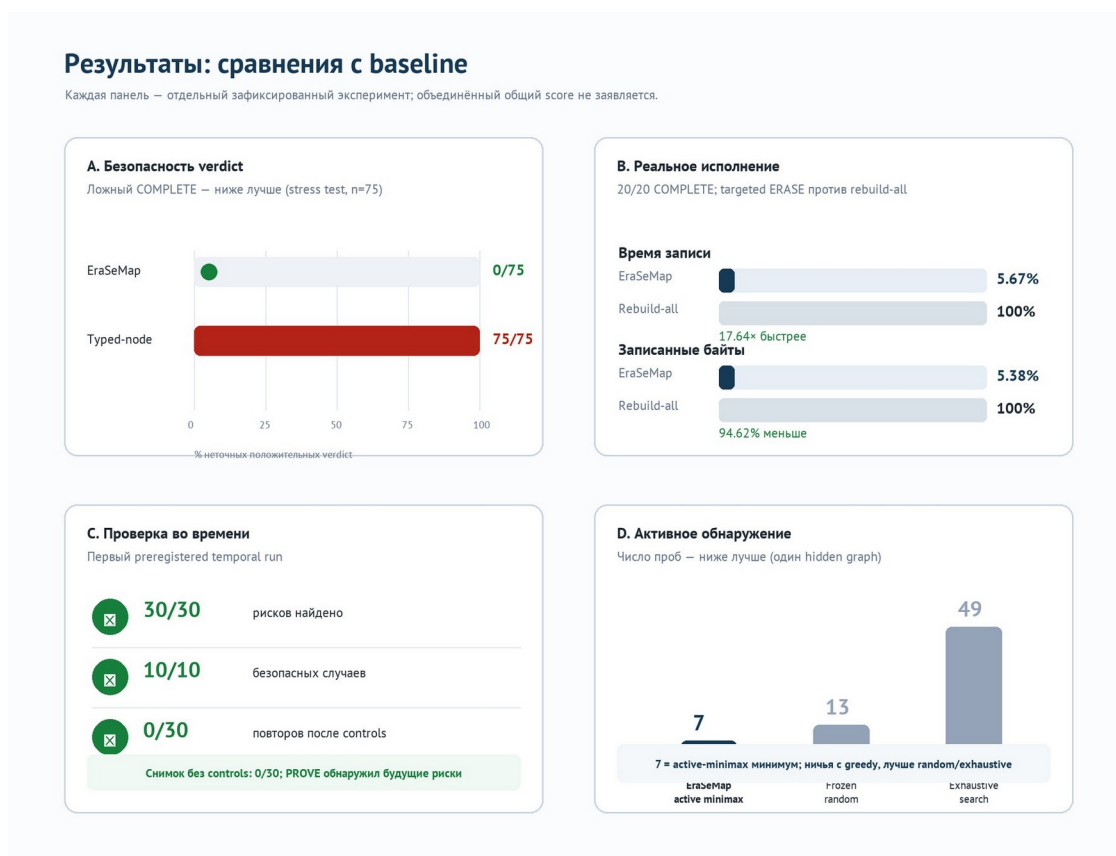


Рисунок 3. Четыре измеренных свойства единого алгоритма EraSeMap: безопасность, эффективность, временная проверка и активное обнаружение. Составлено автором по зафиксированным результатам.

Главный результат этой работы — не то, что мой метод обгоняет какой-то конкретный численный алгоритм. Дело в другом: EraSeMap связывает разные виды доказательств вместе, поэтому одна слабая локальная метрика не может закрыть весь запрос на удаление. Низкий membership score, например, ничего не говорит о том, удалена ли резервная копия. Пустой снимок состояния системы не доказывает, что будущее восстановление не вернёт данные обратно.

Активный поиск в FIND сокращает число проб, но на моём зафиксированном наборе графов жадная стратегия дала точно такой же результат, и я не стал это прятать. Выбранный план в ERASE экономит время по сравнению с полной пересборкой системы, но измерение сделано на одной локальной машине, и на другом железе цифры точно будут другими. PROVE проверяет будущее только там, где переходы системы вообще зарегистрированы — если про какую-то операцию восстановления я не знал заранее,

PROVE её не увидит. У каждой цифры в этой работе есть своя граница применимости, и мне кажется важным говорить об этом прямо, а не только в разделе про ограничения.

10. Ограничения и угрозы валидности

- Скрытые графы, сценарии ошибок и порядок запуска я придумал и настроил сам — независимого внешнего запуска не было.
- Сервисы настоящие (Keycloak, MLflow, Qdrant, PostgreSQL, Redis), но субъекты — синтетические или публичные тестовые данные, а не записи реальных клиентов.
- Список разрешённых действий ограничен — он не покрывает произвольную закрытую инфраструктуру, которую я не тестировал.
- В локальном проверяющем коде теоретически может быть ошибка; цифровая подпись защищает файл от подмены, но не доказывает, что само измерение верно.
- Скорость я измерял на одной локальной конфигурации — на других машинах цифры будут другими.
- Опыт с лицами — это не сертификация приватности в юридическом смысле.
- Эксперименты с Qwen касаются влияния через адаптер, а не удаления из исходного предобучения самой модели.
- Внедрения в FaceID, eGov, KYC, банковские или государственные системы я не проводил и не претендую на это.

11. Этика и ответственное применение

Все публичные демонстрации я делал только на синтетических личностях или открытых наборах данных. Любые тестовые пробы на реальной системе нужно заранее согласовывать с владельцем этой системы, проводить их изолированно и никогда не направлять на данные людей, которые не давали на это согласия. Я специально не хочу, чтобы EraSeMap использовали для формального отчёта «для галочки»: в отчёте по этому алгоритму всегда должны быть видны границы сертификата, неизвестные каналы и пробелы в регистрации системы — иначе сам смысл проверки теряется.

12. Воспроизводимость

```
python -m venv .venv
.venv/bin/pip install -e '[dev,real]'
```

```
./scripts/reproduce_release.sh core  
.venv/bin/erasemap showcase --repo-root . --output outputs/jury-showcase-v1
```

Проверка релиза включает статический анализ (Ruff, strict mypy), тесты pytest с покрытием кода не ниже 90%, сборку проекта, сверку с зафиксированными данными и с полным перебором, а также прогон доказательств в Lean. Отдельный пакет для независимой скрытой проверки я подготовил, но результат по нему пока не собран (NOT_COLLECTED) — честно об этом пишу, а не выдаю желаемое за действительное.

13. Заключение

EraSeMap превращает удаление данных из одной команды в последовательную проверку из трёх шагов. FIND ищет зарегистрированные и ограниченно скрытые пути, по которым данные ещё могут быть доступны. ERASE закрывает физические и модельные каналы самым дешёвым из достаточных планов действий. PROVE проверяет, не вернутся ли данные в будущем, и выдаёт сертификат только в пределах того, что реально проверено.

Формальные доказательства в этой работе зависят от явно указанных предположений, и я сохранил отрицательные результаты по машинному обучению вместо того, чтобы их скрыть. Внедрение в системы вроде FaceID или eGov и независимая внешняя проверка — это следующие шаги, а не то, что уже сделано в этой работе.

Использование инструментов ИИ

Ниже я честно указываю, какую роль играли цифровые инструменты на разных этапах. Для меня важно не скрывать помощь, а показать, какие решения я понимаю, какие результаты проверил и за какие выводы отвечаю сам.

- Идея и постановка задачи: я выбрал проблему неполного удаления данных и сформулировал общий вопрос о копиях, производных, влиянии в модели и будущем восстановлении. Научный руководитель помогал обсуждать понятность и границы работы.
- Код на Python и Lean: при разработке использовались документация, открытые библиотеки и ИИ-ассистент для черновых вариантов, поиска ошибок и объяснения сложных мест. Итоговый код проверяется тестами, отдельными реализациями полного перебора и ядром Lean; я разбирал результаты и могу объяснить основные модули и проверки.

- Эксперименты: запуски автоматизированы скриптами, а протоколы, исходные значения, журналы и результаты сохранены в репозитории. Я запускал проверки, сравнивал выходные данные и оставлял отрицательные результаты, если метод не проходил зафиксированные пороги.
- Текст статьи: структуру, примеры и выводы я выбирал по ходу работы. ИИ-ассистент использовался для редактирования языка, поиска непонятных фрагментов и проверки связности. Финальные формулировки я перечитал и готов объяснить своими словами.
- Формальные доказательства Lean: ИИ-ассистент помогал с черновой формализацией и отладкой. Окончательные доказательства проверяются ядром Lean без недоказанных заглушек; при этом они подтверждают только записанные предположения и не доказывают полноту реальной инфраструктуры.

Я готов подробно объяснить любую часть этой работы на защите, независимо от того, какие инструменты использовались на этапе подготовки.

Список литературы

- [1] Cao Y., Yang J. Towards Making Systems Forget with Machine Unlearning. IEEE S&P, 2015. DOI: 10.1109/SP.2015.35.
- [2] Bourtole L. et al. Machine Unlearning. IEEE S&P, 2021. arXiv:1912.03817.
- [3] Sommer D. M. et al. Towards Probabilistic Verification of Machine Unlearning. arXiv:2003.04247, 2020.
- [4] Weng J. et al. Proof of Unlearning: Definitions and Instantiation. arXiv:2210.11334, 2022.
- [5] Eisenhofer T. et al. Verifiable and Provably Secure Machine Unlearning. SaTML, 2025.
- [6] Chourasia R., Shah N. Forget Unlearning: Towards True Data-Deletion in Machine Learning. ICML, 2023.
- [7] Zhang B. et al. Verification of Machine Unlearning is Fragile. ICML, 2024.
- [8] Koloskova A. et al. Certified Unlearning for Neural Networks. ICML, 2025.
- [9] Lebo T., Sahoo S., McGuinness D., eds. PROV-O: The PROV Ontology. W3C Recommendation, 2013.
- [10] US20220414070A1. Tracking Data Lineage and Applying Data Removal to Enforce Data Removal Policies, 2022.
- [11] US11120156B2. Privacy Preserving Data Deletion, 2021.

[12] US12456052B2. Systems and Methods for Facilitating Verifiability of ML Model Unlearning, 2025.

[13] NIST SP 800-63A-4. Digital Identity Guidelines: Identity Proofing and Enrollment, 2025.

[14] EraSeMap. Публичный репозиторий и evidence archive, v0.5.0, 2026: <https://github.com/nazkari86-lab/erasemap>.

[15] Chakraborty V. et al. Meaningful Data Erasure in the Presence of Dependencies. PVLDB 18(10), 2025.

[16] Regulation (EU) 2016/679 (General Data Protection Regulation), Article 17 «Right to erasure ('right to be forgotten')», 2016.

[17] Закон Республики Казахстан «О персональных данных и их защите» от 21 мая 2013 года № 94-V.

Приложение А. Обозначения

- $G = (V, E, \tau, s)$ — зарегистрированный типизированный граф удаления;
- u — субъект одного запроса на удаление;
- $R_u(G)$ — активные остаточные пути субъекта;
- C_u — обязательные каналы проверки;
- A — конечный каталог допустимых действий;
- B^* — минимальный достаточный набор действий;
- H — ограниченное пространство версий графов восстановления;
- δ — зарегистрированные будущие переходы;
- FCR — доля ложных «удалено» (false-complete rate).

Приложение В. Соответствие утверждений и доказательств

Утверждение	Проверенный результат	Граница вывода
Повторная проверка условно корректна	Теорема Lean и контрпримеры	Нужны полная зарегистрированная топология и корректные локальные проверки
ERASE выбирает минимальный план	Lean и 3072/3072 совпадений с полным перебором	Конечный зарегистрированный каталог действий
PROVE проверяет безопасность во времени	Lean и 16 384/16 384 совпадений с полным перебором	Только зарегистрированные будущие переходы
FIND уменьшает число проб	7 против 13 у случайной стратегии и 49 у полного перебора	Зафиксированный набор проекта; с жадной стратегией ничья
Выбранный ERASE дешевле полной пересборки	Ускорение 17,64× и на 94,62% меньше записанных байтов	Одна локальная система и синтетические личности
Быстрое удаление влияния из Qwen	Не подтверждено: v1 и v2 получили FAIL	Точное переобучение адаптера — безопасный запасной вариант в этом протоколе
Независимая скрытая проверка	Протокол и пакет готовы	Результат пока NOT_COLLECTED
Внедрение в FaceID/eGov	Есть только протокол пилота	Реального внедрения не проводилось